Fundamentals of Data Structures

Prof. Pranjal Pandit

Course Examination Scheme

Syllabus Template FF No.: 654-A

CI2001: Fundamentals of Data Structures

Teaching Scheme	Examination Scheme
Credits :4	CP:30 Marks
Lectures: 3 Hrs/Week	CVV: 20 Marks
Practical: 2 Hrs/Week	Lab: 10 Marks
	ESE :40 Marks

Prerequisites:	Basic programming skills (C/ Python)

Unit 1 Introduction to Algorithms and Data Structures

- From Problem to Program (Problem, Solution, Algorithm, Data Structure and Program).
- Data, Information, Knowledge and Data Structure;
- Abstract Data Types (ADT);
- Classification (Linear/Non-linear, Static/Dynamic, Persistent/Ephemeral).
- Algorithms: problem solving, characteristics, design tools (pseudo-code, flowchart).
- Complexity: space and time complexity, asymptotic notation (Big-O, Theta, Omega),
- Step-count method, analysis of Linear, Quadratic, Cubic and Logarithmic constructs.
- Algorithmic strategies: Divide and Conquer, Greedy.

Unit II – Linear Data Structures Using Sequential Organization (7 Hours)

Concept of sequential organization; arrays as an ADT and array operations; merging of two arrays; storage representation and address calculation (row-major, column-major); multi-dimensional arrays. Ordered lists; single-variable polynomial representation using arrays, polynomial as array of structures, polynomial addition and multiplication. Sparse matrices: representation, addition, simple and fast transpose, time–space trade-off.

Unit III – Searching and Sorting (7 Hours)

Searching: sequential/linear search, sentinel search, binary search, Fibonacci search, indexed sequential search. Sorting: internal vs external sorting; sort order, stability, efficiency and number of passes; comparison-based methods (Bubble, Insertion, Selection, Quick, Shell); non-comparison-based methods (Radix, Counting, Bucket); comparison of all methods and their complexities.

Unit IV – Linked List (7 Hours) Static vs dynamic memory allocation; introduction to linked lists, realization using dynamic memory management, linked list as ADT. Types: singly linked (linear and circular), doubly linked, doubly circular linked lists. Primitive operations: create, traverse, search, insert, delete, sort, concatenate. Polynomial manipulation using linked lists; Generalized Linked List (GLL) concept and polynomial representation using GLL.
Unit V – Stack (7 Hours) Basic concept, stack ADT, representation using sequential organization, stack operations, multiple stacks. Applications: expression evaluation and conversion, Polish notation, need for prefix/postfix expressions, postfix expression evaluation, linked stack and operations. Recursion: concept and variants (direct, indirect, tail, tree); backtracking strategy and use of stack in backtracking.
Unit VI – Queue (7 Hours) Basic concept, queue as ADT, representation using sequential organization, queue operations, circular queue and its advantages, multi-queues, linked queue and operations. Deque: basic concept, input-restricted and output-restricted types. Priority queue: basic concept, ascending and descending types.

Topic 1: From Problem to Program (Problem → Solution → Algorithm → Data Structure → Program)

Introduction

- Every software application begins with a **problem** that needs to be solved.

- A programmer's task is not simply to write code but to understand the problem, develop a systematic solution, design an efficient algorithm, choose appropriate data structures, and finally implement the solution as a program.

- For example, consider an **online shopping application**. The application must search products, add items to the cart, process payments, and generate bills. These tasks involve solving different computational problems efficiently.

Thus, software development follows a logical sequence, where each stage build upon the previous one:

Problem: A **problem** is a task or requirement that needs to be solved to achieve a desired objective.

In computing, a problem specifies:

- the **input** provided to the system,
- the **processing** to be performed, and
- the **expected output**.

Example

Problem: Find the highest marks among 60 students.

Input: Marks of 60 students.

Output: Highest mark.

Real-Life Examples

Problem	Desired Output
Find shortest route	Best path
Search a contact	Contact details
Withdraw money	Cash
Calculate salary	Net salary

Solution

Definition

- A **solution** is the method or approach used to solve a problem.
- There can be multiple solutions to the same problem.

The best solution is one that

- is correct,
- requires less time,
- uses less memory,
- is simple to understand.

Example

Problem: Arrange students according to marks.

Possible solutions:

- Bubble Sort
- Selection Sort
- Merge Sort
- Quick Sort

All solve the same problem, but some are much more efficient than others.

Algorithm: An algorithm is a **finite sequence of well-defined instructions** that solves a problem and produces the desired output. An algorithm describes **what should be done**, not **how it is coded**.

Example: Find the largest of three numbers.

Algorithm

- Step 1: Read A, B, C

  Step 2: If A ≥ B and A ≥ C
  Largest = A

  Step 3: Else if B ≥ C
  Largest = B

  Step 4: Else
  Largest = C

  Step 5: Display Largest

  Step 6: Stop

Characteristics of a Good Algorithm

1. **Input:** Accepts zero or more inputs.

2. **Output** Produces at least one output.

3. **Definiteness:** Every instruction must be precise and unambiguous.

4. **Finiteness:** Terminates after a finite number of steps.

5. **Effectiveness:** Each instruction should be basic and executable.

6. **Correctness:** Produces correct results for all valid inputs.

Need for algorithms

1. Efficiency and Optimization
2. Automation of Tasks
3. Problem solving
4. Consistency and Accuracy
5. Foundation of Computer Science
6. Handling complex systems
7. Problem generalization

Data Structure

Definition

- A data structure is a **systematic way of organizing and storing data** so that it can be accessed and modified efficiently.
- Data structures improve the performance of algorithms.

Example: Suppose we want to search for a student's roll number.

Possible storage methods:

- Unorganized list
- Array
- Linked List
- Binary Search Tree
- Hash Table

Choosing the right data structure can reduce the search time significantly.

Why Do We Need Data Structures?

Without proper organization:

- Searching becomes slow.
- Insertion becomes difficult.
- Deletion becomes inefficient.
- Programs consume more memory.

Real-Life Analogy

Real Life	Data Structure
Books arranged on shelves	Array/Stack
Queue at a ticket counter	Queue
Stack of plates	Stack
Family tree	Tree
Road network	Graph

Program: A **program** is the implementation of an algorithm in a programming language so that a computer can execute it. A program converts the algorithm into machine-executable instructions.

Program (C): add 2 numbers

```
#include <stdio.h>

int main()
{
    int a, b, sum;

    printf("Enter two numbers: ");
    scanf("%d%d", &a, &b);

    sum = a + b;

    printf("Sum = %d", sum);

    return 0;
}
```

Relationship among Problem, Algorithm, Data Structure and Program

Component	Purpose
Problem	Defines what needs to be solved
Solution	Overall approach to solving the problem
Algorithm	Step-by-step procedure
Data Structure	Organizes data efficiently
Program	Implementation of the algorithm in a programming language

Complete Example

Problem: Store marks of 100 students and find the highest mark.

↓

Solution: Store all marks and compare them one by one.

↓

Algorithm

- Read marks.
- Store marks.
- Compare each mark.
- Find maximum.
- Display maximum.

↓

Data Structure: Array

↓

Program: C program using an array and a loop.

FROM PROBLEM TO PROGRAM

The Foundation of Computer Programming and Data Structures

1 PROBLEM



A real-world task or question that needs to be solved.

2 SOLUTION



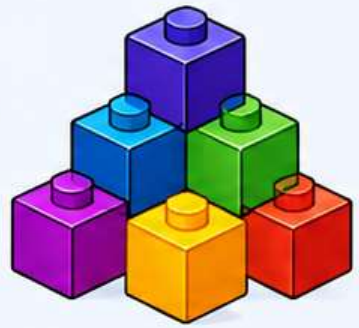
An approach or method to solve the problem.

3 ALGORITHM



A finite sequence of well-defined steps to solve the problem.

4 DATA STRUCTURE



A way of organizing and storing data efficiently to support the algorithm.

5 PROGRAM



Implementation of the algorithm in a programming language that the computer can execute.

Problem → Solution → Algorithm → Data Structure → Program

This sequence forms the foundation of computer programming and data structures.

Why Algorithms and Data Structures Are Studied Together

An efficient algorithm alone is not enough. It also requires an efficient way to organize data.

For example, searching for a student in:

- an unsorted array may require checking every element,
- a sorted array allows binary search,
- a hash table can often locate the record almost instantly.

Thus,

Algorithm + Appropriate Data Structure = Efficient Program

This idea is the foundation of the subject *Fundamentals of Data Structures*.

Key Differences

Problem	**Algorithm**	**Program**
Requirement	Procedure	Implementation
Describes what is needed	Describes how to solve	Executes on computer
Independent of language	Independent of language	Language dependent
Conceptual	Logical	Practical

Which of the following is the correct sequence in software development?

A) Problem → Program → Algorithm → Data Structure → Solution

B) Problem → Solution → Algorithm → Data Structure → Program

C) Solution → Problem → Data Structure → Algorithm → Program

D) Algorithm → Solution → Program → Data Structure → Problem

Which of the following is the correct sequence in software development?

A) Problem → Program → Algorithm → Data Structure → Solution

B) Problem → Solution → Algorithm → Data Structure → Program

C) Solution → Problem → Data Structure → Algorithm → Program

D) Algorithm → Solution → Program → Data Structure → Problem

A problem in computing is best defined as:

A) A programming language

B) A set of computer instructions

C) A task or requirement that needs to be solved

D) A collection of data

A problem in computing is best defined as:

A) A programming language

B) A set of computer instructions

C) A task or requirement that needs to be solved

D) A collection of data

An algorithm is:

A) A computer program written in C

B) A finite sequence of well-defined steps to solve a problem

C) A programming language

D) A compiler

An algorithm is:

A) A computer program written in C

B) A finite sequence of well-defined steps to solve a problem

C) A programming language

D) A compiler

Which characteristic of an algorithm ensures that it eventually stops?

A) Input

B) Output

C) Finiteness

D) Correctness

Which characteristic of an algorithm ensures that it eventually stops?

A) Input

B) Output

C) Finiteness

D) Correctness

A data structure is primarily used to:

A) Increase the size of memory

B) Organize and store data efficiently

C) Write computer programs

D) Translate source code

A data structure is primarily used to:

A) Increase the size of memory

B) Organize and store data efficiently

C) Write computer programs

D) Translate source code

Which of the following is language dependent?

A) Problem

B) Algorithm

C) Solution

D) Program

Which of the following is language dependent?

A) Problem

B) Algorithm

C) Solution

D) Program

Which statement is TRUE regarding an algorithm?

A) It must be written only in C language.

B) It is independent of any programming language.

C) It can execute directly on a computer.

D) It always contains program statements.

Which statement is TRUE regarding an algorithm?

A) It must be written only in C language.

B) It is independent of any programming language.

C) It can execute directly on a computer.

D) It always contains program statements.

Which component converts an algorithm into executable instructions?

A) Problem

B) Data Structure

C) Program

D) Solution

Which component converts an algorithm into executable instructions?

A) Problem

B) Data Structure

C) Program

D) Solution

Choosing an appropriate data structure mainly helps to:

A) Increase the size of the program

B) Improve the efficiency of operations

C) Reduce the number of algorithms

D) Eliminate the need for programming

Choosing an appropriate data structure mainly helps to:

A) Increase the size of the program

B) Improve the efficiency of operations

C) Reduce the number of algorithms

D) Eliminate the need for programming

Which of the following is NOT a characteristic of a good algorithm?

A) Definiteness

B) Finiteness

C) Ambiguity

D) Effectiveness

Which of the following is NOT a characteristic of a good algorithm?

A) Definiteness

B) Finiteness

C) Ambiguity

D) Effectiveness

A programmer has designed an excellent algorithm, but the program still runs very slowly for large amounts of data. What is the most likely reason?

A) The algorithm is incorrect.

B) An inappropriate data structure has been used.

C) The programming language is too simple.

D) The computer monitor is slow.

A programmer has designed an excellent algorithm, but the program still runs very slowly for large amounts of data. What is the most likely reason?

A) The algorithm is incorrect.

B) An inappropriate data structure has been used.

C) The programming language is too simple.

D) The computer monitor is slow.

Explanation: An efficient algorithm combined with an inefficient data structure can significantly degrade performance. Selecting the right data structure is essential for achieving optimal execution time and memory usage.

FROM PROBLEM TO PROGRAM

Problem → Solution → Algorithm → Data Structure → Program

1. PROBLEM

A real-world task that needs to be solved.



2. SOLUTION

A method or approach to solve the problem.



3. ALGORITHM

A finite sequence of well-defined steps to solve the problem.



4. DATA STRUCTURE

A way of organizing and storing data efficiently.



5. PROGRAM

Implementation of the algorithm in a programming language.



THE JOURNEY

- Real-World Problem
- Understand the Problem
- Develop a Solution
- Design an Algorithm
- Choose Suitable Data Structure
- Write the Program
- Execute and Obtain Results

COMPLETE EXAMPLE

PROBLEM — Find the highest marks among 5 students.



SOLUTION — Store all marks and compare them one by one.

ALGORITHM

1. Read marks.
2. Store marks.
3. Compare each mark.
4. Find maximum.
5. Display maximum.

DATA STRUCTURE — Array

PROGRAM — C program using array and loop.

WHY DATA STRUCTURE?

Choosing the right data structure makes operations faster and memory efficient.

WITHOUT GOOD DS	WITH GOOD DS
Operations are slow	Operations are fast
Poor organization	Better organization
More memory consumption	Efficient memory usage

REAL-LIFE ANALOGY

Books on a shelf	Array
Queue at a counter	Queue
Stack of plates	Stack
Family tree	Tree
Road network	Graph

DIFFERENCE BETWEEN

Aspect	Algorithm	Program
What it is	Step-by-step procedure	Implementation of algorithm
Language	Independent of programming language	Dependent on programming language
Purpose	Describes how to solve the problem	Executed by computer
Nature	Logical	Physical (Code)
Focus	What to do?	How to do?

KEY TAKEAWAY

- Start with understanding the problem.
- Think of possible solutions.
- Design an algorithm (step-by-step).
- Choose the best data structure.
- Implement the algorithm as a program.
- Test, execute and get the results.

Efficient Program = Good Algorithm + Appropriate Data Structure

Data, Information, Knowledge

Understanding the difference between Data, Information, and Knowledge (DIK) is essential because **every computer system processes data** to **generate information**, which **helps people gain knowledge** and make decisions.

For example, when students write an examination, they obtain marks in different subjects. These marks by themselves are just **numbers (data)**. When the marks are processed to calculate the total and percentage, they become **information**. When teachers analyze this information to identify the strengths and weaknesses of students, they gain **knowledge**.

Thus,

Data → Information → Knowledge

This transformation helps organizations, businesses, and individuals make better decisions.

Data

Definition

- **Data** is a collection of **raw, unorganized facts, figures, symbols, observations, or values** that have **no specific meaning** when considered individually.
- Data is the basic input to a computer system.

Characteristics of Data

- Raw and unprocessed
- May be numbers, text, images, or symbols
- Has little or no meaning by itself
- Used as input for processing

Real-Life Example 1 (Student Marks)

65, 78, 82, 91, 56

These are just numbers. Without context, we do not know:

- Whose marks are these?
- Which subject?
- Out of how many marks? Hence, these numbers are **Data**.

Real-Life Example 2 (Weather)

38
42
36
40

These are simply numbers. We do not know whether they represent:

- Temperature
- Age
- Weight
- Speed

Hence, they are raw data.

Information

Definition

- **Information** is **processed, organized, and meaningful data** that is useful for understanding or making decisions.
- Information is obtained after processing data.

Characteristics of Information

- Processed data
- Organized and meaningful
- Easy to understand
- Useful for decision making

Real-Life Example (Weather)

Raw Data

38
42
36
40

Processed Information

Today's Temperatures

8 AM → 36°C

12 PM → 42°C

4 PM → 40°C

Average Temperature = 39.3°C

Now the data has meaningful. This is information.

Knowledge

Definition

- **Knowledge** is the understanding or insight gained by analyzing information along with experience and judgment. It helps in making informed decisions.
- Knowledge answers the question:

"What should we do?"

Characteristics of Knowledge

- Based on information
- Includes experience and understanding
- Helps in decision making
- Can be applied to solve problems

Real-Life Example (Weather)

Information:

Temperature = 42°C

Humidity = High

Weather = Sunny

Knowledge: You decide to:

- Carry a water bottle.
- Wear light cotton clothes.
- Avoid going out in the afternoon.

This decision is based on knowledge.

Comparison Table

Feature	Data	Information	Knowledge
Meaning	Raw facts	Processed data	Understanding gained from information
Processing	Not processed	Processed	Analyzed and interpreted
Context	No context	Has context	Based on context and experience
Purpose	Input	Understanding	Decision making
Example	85, 90, 76	Average = 83.7	Student is performing well and can be recommended for advanced projects.